Twenty-One Years: The Arc of Networking in C++

Document Number:	P4065R0
Date:	2026-03-16
Audience:	WG21

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

R0: March 2026 (post-Croydon mailing)

1. Disclosure

2. The Causal Chain

3. What the Committee Got Right

4. What Is Now Available

5. The Design Fork

6. Anticipated Objections

Acknowledgments

References

Abstract

Four decisions, each locally reasonable, each under-evidenced, produced a decade without networking in the C++ standard.

This paper assembles the findings of five companion papers into a single causal chain: the unification of executors (2014, P4060R0[1]), the basis-operation pivot (2019, P4061R0[2]), the P2464 diagnosis (2021, P4062R0[3]), the networking claim in the poll (2021, P4063R0[4]), and the evidence base for the trajectory

(P4064R0[5]). Each paper documents one decision point. This paper places them in sequence, documents what is now available that was not available when those decisions were made, and credits the work that produced the tools the committee now has. It is the sixth and final paper in the series.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

---

1. Disclosure

The author developed and maintains Corosio[6] and Capy[7] and believes coroutine-native I/O is the correct foundation for networking in C++. Coroutine-native I/O does not provide the sender composition algebra - `retry`, `when_all`, `upon_error` - that `std::execution` provides. The author provides information, asks nothing, and serves at the pleasure of the chair.

The author is revisiting the historical record systematically. This paper is one of several. The goal is to document - precisely and on the record - the decisions that kept networking out of the C++ standard. That effort requires re-examining consequential papers, including papers written by people the author respects.

---

2. The Causal Chain

Year	Decision	Rationale	Published evidence	Companion paper
2014	Three independent executor models unified into one	Shared execution contexts; N x M explosion; synchronization coherence	One hypothetical code snippet. No prototype. No survey. No deployment data showing friction from separate models.	P4060R0[1]

Year	Decision	Rationale	Published evidence	Companion paper
2019	`execute(F&&)` diagnosed as deficient; Cologne pivot to Sender/Receiver	No error channel, no cancellation signal, no zero-allocation scheduling	Diagnosis under the work framing only. Continuation framing not examined. Networking use case not analyzed.	P4061R0[2]
2021	P2464R0 diagnosis applied to Networking TS; networking set aside	Executor is a "work-submitter"; no error channel; no generic composition	Analysis under the work framing only. `async_result` and N3747 not addressed. Deployed compositions not examined.	P4062R0[3]
2021	Poll: sender/receiver "a good basis... including networking" - consensus in favor	P2300 deployments at Facebook, NVIDIA, Bloomberg	Deployments are for GPU dispatch, thread pools, and infrastructure. No sender-based networking deployment. No prototype. One hypothetical example.	P4063R0[4]
2014-2026	Twenty claims shaped the trajectory; evidence documented where it exists	Various	Evidence concentrated in GPU/infrastructure domains. Networking evidence column empty for most claims.	P4064R0[5]

Each decision was locally reasonable. Each was made by experienced practitioners under real constraints. No decision was careless. The record is now complete.

3. What the Committee Got Right

P2300R10[8], "std::execution," is a genuine achievement. The sender/receiver model provides structured concurrency, sender composition, completion signatures as type-level contracts, and a customization point model that enables heterogeneous dispatch. P2470R0[9] documented deployments at Facebook ("monthly users number in the billions"), NVIDIA ("fully invested in P2300... we plan to ship in production"), and Bloomberg (experimentation). GPU dispatch, infrastructure, HPC - the domains where compile-time work graphs, zero-allocation pipelines, and heterogeneous composition deliver their full value.

The people who built this - Eric Niebler, Kirk Shoop, Lewis Baker, Lee Howes, Michał Dominiak, and their collaborators - identified genuine structural problems in the executor concept and proposed a design that solved them. P1525R0[10] documented real deficiencies in `execute(F&&)` under the work framing. The sender/receiver model addressed all four. The committee adopted it. It shipped.

The unification effort that preceded P2300 - P0443[11], more than 100 papers, organizations spanning Google, NVIDIA, Sandia, Codeplay, Facebook, Microsoft, and RedHat - was a sustained, genuine effort to find common ground. The breadth of participation was extraordinary. The compromise was real.

These are facts. This series documents them alongside the facts about what was not examined. Both belong in the record.

---

4. What Is Now Available

The following capabilities exist in 2026 that did not exist when the decisions in Section 2 were made.

C++20 coroutines were ratified in 2020. They did not exist in 2014 when the unification decision was made, or in 2019 when P1525R0[10] diagnosed the basis operation. The coroutine executor concept constrains the handle type to `coroutine_handle<>`, restoring the type constraint that the rename from `dispatch` / `post` / `defer` to `execute(F&&)` removed.

The coroutine executor concept was published in P4003R0[12] (2026). It provides `dispatch(coroutine_handle<>)` and `post(coroutine_handle<>)` - continuation-scheduling primitives with a typed handle. The four deficiencies P1525R0[10] identified do not arise under this concept (P4061R0[2] Section 4).

The two-framing distinction - work framing vs. continuation framing - was documented in P4060R0[1] Section 6. The continuation framing was the original framing (P0113R0[13], 2015). The work framing replaced it through two stages of simplification. No published paper in the causal chain discussed the shift. The distinction is now available for the committee to apply.

The rationale-loss mechanism was documented in P4060R0[1] Section 6.3. API surfaces transfer between papers; design rationale does not, unless someone actively carries it forward. The continuation framing was carried by institutional knowledge rather than by the type system. When the property hint was removed by authors who did not carry that knowledge forward, the framing dropped out. This is a structural property of multi-author standardization, not a criticism of any individual.

Interop bridges between the coroutine model and the sender model were published in P4055R0[14] and P4056R0[15]. The two models can coexist and interoperate.

`std::execution::task` (P3552R3[16]) is simultaneously a coroutine and a sender. The committee has already voted to ship a type that fuses two async models into one. The price of two models has been paid.

---

5. The Design Fork

Awaitable

```
struct read_awaitable
{
    bool await_ready();
    void await_suspend(
        std::coroutine_handle<> h);
        // caller erased
    io_result<size_t>
        await_resume();
};
```

Sender

```
template<class Receiver>
struct read_operation
{
    Receiver rcvr_;
        // caller stamped in
    void start() noexcept;
};
```

`coroutine_handle<>` erases the caller. `connect(sender, receiver)` stamps the caller into the operation state. The first choice produces type-erased streams, separate compilation, and ABI stability - properties that have been difficult to achieve in twenty years of networking attempts. The second produces full pipeline visibility, zero-allocation composition, and compile-time work graphs - the properties that make `std::execution` valuable for GPU dispatch, heterogeneous execution, and infrastructure. Neither model can acquire the other's properties without surrendering its own. The technical analysis is in P4058R0[17].

The committee has already voted to ship `std::execution::task` (P3552R3[16]), a type that is simultaneously a coroutine and a sender. The price of two async models has been paid. The question is whether both models should carry I/O facilities that exploit their respective strengths.

---

6. Anticipated Objections

Q: This is hindsight bias.

A: The companion papers document what evidence was available at the time each decision was made. The questions in Sections 4 and 5 of P4060R0[1] could have been asked in 2014. They were not. The two-framing analysis in P4061R0[2] could not have been performed in 2019 because C++20 coroutines did not yet exist. This paper distinguishes between the two cases.

Q: P2300 will eventually cover networking.

A: It might. The record documents what has shipped as of 2026. No sender-based networking deployment has been published. The committee now has two models to evaluate. Time will tell which serves networking better, or whether both do.

Q: Two models fragment the ecosystem.

A: The committee has shipped multiple design approaches for the same domain before: `iostream` and `std::format` / `std::print`; C++17 execution policies and P2300 senders with `bulk`. `std::execution::task` (P3552R3[16]) is simultaneously a coroutine and a sender. The committee has already chosen coexistence.

Q: You are arguing for your own library.

A: Section 1 discloses this. The causal chain in Section 2 is assembled from the published record. The evidence stands or falls on the sources, not on who assembled them.

Acknowledgments

The author thanks Chris Kohlhoff for the executor model that started the journey, for twenty years of Boost.Asio deployment, for the candid retrospective in P1791R0[18], and for N3747[19]'s universal async model; Eric Niebler, Kirk Shoop, Lewis Baker, and Lee Howes for the sender/receiver model, for P1525R0[10]'s genuine structural insights, and for P2300[8]; Michał Dominiak for the editorial work that brought P2300R10[8] to adoption; Jared Hoberock, Michael Garland, and Chris Mysen for P0443[11], P0761R2[20], and years of compromise in pursuit of a unified model; Ville Voutilainen for P2464R0[21]'s analytical framework, which provided the evaluation criteria this series applies; Bryce Adelstein Lelbach for the published poll outcomes in P2453R0[22] and P2400R2[23] that made this analysis possible; Dietmar Kühl for P2762R2[24] and `beman::execution`; Detlef Vollmann for P1256R0[25]; Jonathan Müller for P3801R0[26]; Peter Dimov for technical corrections on earlier drafts; Steve Gerbino and Mungo Gill for Capy[7] and Corosio[6] implementation work; Klemens Morgenstern for Boost.Cobalt and the cross-library bridge examples; Jamie Allsop and Richard Hodges for co-authoring P2469R0[27]; and the national body members who raised concerns at St. Louis.

The effort to bring async programming to C++ has been genuine, sustained, and conducted by people the author respects. This series documents the record. The committee will decide what to do with it.

References

1. P4060R0 - "Retrospective: The Unification of Executors and P0443" (Vinnie Falco, 2026). https://wg21.link/p4060r0
2. P4061R0 - "Retrospective: The Basis Operation and P1525" (Vinnie Falco, 2026). https://wg21.link/p4061r0
3. P4062R0 - "Retrospective: Coroutine Executors and P2464R0" (Vinnie Falco, 2026). https://wg21.link/p4062r0
4. P4063R0 - "Retrospective: The Networking Claim and P2453R0" (Vinnie Falco, 2026). https://wg21.link/p4063r0
5. P4064R0 - "Retrospective: Claims and Evidence" (Vinnie Falco, 2026). https://wg21.link/p4064r0
6. cppalliance/corosio - Coroutine-native networking library. https://github.com/cppalliance/corosio
7. cppalliance/capy - Coroutine I/O primitives library. https://github.com/cppalliance/capy
8. P2300R10 - "std::execution" (Michał Dominiak et al., 2024). https://wg21.link/p2300r10
9. P2470R0 - "Slides for presentation of P2300R2" (Eric Niebler, 2021). https://wg21.link/p2470r0
10. P1525R0 - "One-Way execute is a Poor Basis Operation" (Eric Niebler, Kirk Shoop, Lewis Baker, Lee Howes, 2019). https://wg21.link/p1525r0
11. P0443R0 - "A Unified Executors Proposal for C++" (Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysen, Carter Edwards, 2016). https://wg21.link/p0443r0
12. P4003R0 - "Coroutines for I/O" (Vinnie Falco, Steve Gerbino, Mungo Gill, 2026). https://wg21.link/p4003r0
13. P0113R0 - "Executors and Asynchronous Operations, Revision 2" (Christopher Kohlhoff, 2015). https://wg21.link/p0113r0
14. P4055R0 - "Consuming Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/p4055r0
15. P4056R0 - "Producing Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/p4056r0
16. P3552R3 - "Add a Coroutine Task Type" (Dietmar Kühl, Maikel Nadolski, 2025). https://wg21.link/p3552r3
17. P4058R0 - "The Case for Coroutines" (Vinnie Falco, 2026). https://wg21.link/p4058r0

18. P1791R0 - “Evolution of the P0443 Unified Executors Proposal to accommodate new requirements” (Christopher Kohlhoff, Jamie Allsop, 2019). https://wg21.link/p1791r0

19. N3747 - “A Universal Model for Asynchronous Operations” (Christopher Kohlhoff, 2013). https://wg21.link/n3747

20. P0761R2 - “Executors Design Document” (Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysen, Carter Edwards, Gordon Brown, Michael Wong, 2018). https://wg21.link/p0761r2

21. P2464R0 - “Ruminations on networking and executors” (Ville Voutilainen, 2021). https://wg21.link/p2464r0

22. P2453R0 - “2021 October Library Evolution Poll Outcomes” (Bryce Adelstein Lelbach, Fabio Fracassi, Ben Craig, 2022). https://wg21.link/p2453r0

23. P2400R2 - “Library Evolution Report: 2021-06-01 to 2021-09-20” (Bryce Adelstein Lelbach, 2021). https://wg21.link/p2400r2

24. P2762R2 - “Sender/Receiver Interface For Networking” (Dietmar Kühl, 2023). https://wg21.link/p2762r2

25. P1256R0 - “Executors Should Go To A TS” (Detlef Vollmann, 2018). https://wg21.link/p1256r0

26. P3801R0 - “Concerns about the design of std::execution::task” (Jonathan Müller, 2025). https://wg21.link/p3801r0

27. P2469R0 - “Response to P2464: The Networking TS is baked, P2300 Sender/Receiver is not” (Christopher Kohlhoff, Jamie Allsop, Vinnie Falco, Richard Hodges, Klemens Morgenstern, 2021). https://wg21.link/p2469r0